

Museum Heist: Thief vs Cooperating Guards using Minimax

Parth Pathak

Department of Computer Science and Engineering

Amrita School of Computing

Amrita Vishwa Vidyapeetham

Bengaluru, India

bl.en.u4cse23036@bl.students.amrita.edu

Abstract—This paper presents a multi-agent simulation of a museum heist in which a thief tries to steal an artifact and reach an exit while two guards, acting as a cooperating team, try to catch the thief first. The thief and the guards are modelled as utility based agents that use depth limited Minimax search with Alpha-Beta pruning over a grid world. The system is evaluated on four required scenarios, two working cases and two edge cases, and is compared against a one-step greedy search and against a version where the two guards search independently instead of as a team. Results show that Alpha-Beta pruning reduces the number of nodes expanded by more than half at deeper search levels without changing the chosen move, that a shallow search can miss a threat that a deeper search would have avoided (the horizon effect), and that splitting one search tree across a team of agents reduces the effective lookahead available to each of them compared to independent search at the same nominal depth.

Index Terms—multi-agent systems, adversarial search, minimax, alpha-beta pruning, artificial intelligence

I. INTRODUCTION

Adversarial search is one of the core problems studied in artificial intelligence, and it becomes more interesting once more than two agents are involved and some of them must work together while opposing a common target. This case study designs and analyzes a small multi-agent system called Museum Heist. A thief agent must locate an artifact inside a museum, pick it up, and then reach an exit before being caught. Two guard agents patrol the museum and cooperate with each other to intercept the thief, while competing against it.

The purpose of this study is not simply to build a working simulation, but to demonstrate and explain the underlying AI concepts: how a PEAS description and environment properties shape the choice of agent type, how depth limited Minimax search with Alpha-Beta pruning lets an agent make good decisions without exploring an entire game tree, and how the depth of that search directly affects the quality and speed of the decisions made. The system is tested on four required cases and compared against two alternative approaches, a greedy search with no lookahead, and a version of the guard team that searches independently rather than as a coordinated unit.

II. PROBLEM DOMAIN AND STATEMENT

The problem domain is a grid based museum environment. The museum is represented as a two dimensional grid con-

taining walls, open floor cells, an artifact, and an exit. Three agents occupy this grid: one thief and two guards.

The thief's goal is to reach the artifact cell, pick it up, and then reach the exit cell without being caught by a guard. A guard catches the thief by moving onto the same cell as the thief. The guards' goal is to prevent the thief from escaping with the artifact, either by capturing it directly or by keeping it away from the exit until a turn limit is reached.

Agents move in turns, in the fixed order thief, guard one, guard two, and each agent can move up, down, left, right, or wait in place, as long as the destination cell is not a wall. This turn structure keeps the branching factor small and predictable, which makes the search tree easy to construct, trace, and explain.

III. PEAS DESCRIPTION

Table I summarizes the Performance measure, Environment, Actuators, and Sensors for the thief agent. The guard agents share the same environment, actuators, and sensors, but their performance measure is defined in the opposite direction, rewarded for capturing the thief and for keeping it away from the artifact and the exit.

TABLE I
PEAS DESCRIPTION FOR THE THIEF AGENT

Component	Description
Performance measure	Reaching the artifact, then reaching the exit, while avoiding capture by a guard
Environment	Grid based museum with walls, an artifact cell, an exit cell, and two guards
Actuators	Move up, down, left, right, or wait
Sensors	Own position, positions of both guards, artifact and exit locations, and the layout of walls in the grid

IV. ENVIRONMENT PROPERTIES

- **Observable:** The environment is fully observable. Every agent has access to the full grid layout and the exact positions of every other agent at every turn.
- **Deterministic:** The environment is deterministic. A chosen move always produces exactly one resulting position, with no randomness involved.

- **Episodic or sequential:** The environment is sequential. Each move affects future states, and an agent's current decision depends on the outcomes of its earlier decisions.
- **Static or dynamic:** The environment is static between turns. Nothing changes on the grid unless an agent moves, and agents move one at a time in a fixed order.
- **Discrete or continuous:** The environment is discrete in both space, since positions are grid cells, and in time, since moves happen in distinct turns.
- **Multi-agent:** The environment is multi-agent, with three agents present at once. It is partly cooperative, since the two guards work together, and partly competitive, since the guards oppose the thief.

V. AGENT TYPE AND INTERACTION MODEL

All three agents are utility based agents. Each agent evaluates the possible future states reachable within a limited number of moves and selects the action that leads towards the state with the best score, according to its own goals. A utility based agent was chosen over a simpler goal based agent because the environment involves an opponent, so an agent needs to compare states by how favorable they are relative to a moving adversary, not simply check whether a fixed goal condition is met.

The interaction between agents is both cooperative and competitive. The two guards are cooperative with each other, since they are modelled as MIN nodes inside one shared search tree, which means each guard's search already accounts for what the other guard is expected to do. At the same time, both guards are competitive with the thief, which is modelled as the MAX node in that same tree. This produces a three agent adversarial search where one side has a coordination advantage in principle, and Section IX examines how that advantage depends on the available search depth.

VI. SEARCHING TECHNIQUE

A. Minimax

The core searching technique used is depth limited Minimax. The thief is treated as the MAX player, trying to maximize an evaluation score, while both guards are treated as MIN players, trying to minimize the same score. The search proceeds turn by turn in the fixed order thief, guard one, guard two, building a tree of possible future states up to a fixed depth limit. At the depth limit, or when the game reaches a terminal state such as a capture or an escape, a heuristic evaluation function scores the position, and these scores are propagated back up the tree, with MAX levels keeping the highest score and MIN levels keeping the lowest.

Algorithm 1 shows the structure of this search.

B. Evaluation function

The evaluation function scores a state using shortest path distances, precomputed once per map with breadth first search from every walkable cell. If the artifact has not yet been collected, the thief's goal distance is the distance from the thief to the artifact, otherwise it is the distance from the thief

Algorithm 1 Depth limited Minimax

```

1: function MINIMAX(state, depth)
2:   if state is terminal or depth is 0 then
3:     return Evaluate(state)
4:   end if
5:   if state.turn is Thief then
6:     bestScore  $\leftarrow -\infty$ 
7:     for each legal move of the thief do
8:       child  $\leftarrow$  ApplyMove(state, move)
9:       score  $\leftarrow$  MINIMAX(child, depth - 1)
10:      bestScore  $\leftarrow$  max(bestScore, score)
11:    end for
12:    return bestScore
13:  else
14:    bestScore  $\leftarrow +\infty$ 
15:    for each legal move of the current guard do
16:      child  $\leftarrow$  ApplyMove(state, move)
17:      score  $\leftarrow$  MINIMAX(child, depth - 1)
18:      bestScore  $\leftarrow$  min(bestScore, score)
19:    end for
20:    return bestScore
21:  end if
22: end function

```

to the exit. The score is the distance from the nearest guard to the thief, minus the thief's goal distance, so that the thief favors states where guards are far away and its own goal is close. Terminal states are given large fixed scores, a positive score for the thief reaching the exit with the artifact, and a large negative score for the thief being captured.

C. Alpha-Beta pruning

Alpha-Beta pruning is used alongside Minimax to skip branches of the search tree that cannot influence the final decision, without changing the value returned or the move chosen. Two bounds, alpha and beta, are tracked during the search, and a branch is pruned once it is shown that the current player would never allow that branch to be reached. Move ordering, trying moves that reduce distance to the relevant goal first, is used to increase the number of branches pruned early. Algorithm 2 shows the modified search.

D. Search tree example

Figure 1 shows a small example of the search tree at the very first move of a game, expanded two plies deep. The thief chooses among its legal moves at the MAX level, and the guard responds at the MIN level. Branches that Alpha-Beta pruning would skip, because they cannot change the parent's decision, are marked with a dashed border.

VII. WORKING CASES AND EDGE CASES

Four required scenarios were built and tested, each on its own map, with results generated by running the full simulation to completion.

Algorithm 2 Minimax with Alpha-Beta pruning

```

1: function ALPHABETA(state, depth, alpha, beta)
2:   if state is terminal or depth is 0 then
3:     return Evaluate(state)
4:   end if
5:   if state.turn is Thief then
6:     bestScore  $\leftarrow -\infty$ 
7:     for each legal move, best first order do
8:       child  $\leftarrow$  ApplyMove(state, move)
9:       score  $\leftarrow$  ALPHABETA(child, depth - 1, alpha,
beta)
10:      bestScore  $\leftarrow$  max(bestScore, score)
11:      alpha  $\leftarrow$  max(alpha, bestScore)
12:      if beta  $\leq$  alpha then
13:        break
14:      end if
15:    end for
16:    return bestScore
17:   else
18:     bestScore  $\leftarrow +\infty$ 
19:     for each legal move, best first order do
20:       child  $\leftarrow$  ApplyMove(state, move)
21:       score  $\leftarrow$  ALPHABETA(child, depth - 1, alpha,
beta)
22:      bestScore  $\leftarrow$  min(bestScore, score)
23:      beta  $\leftarrow$  min(beta, bestScore)
24:      if beta  $\leq$  alpha then
25:        break
26:      end if
27:    end for
28:    return bestScore
29:   end if
30: end function

```

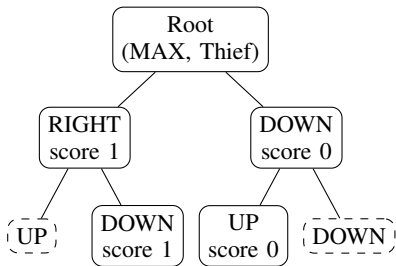


Fig. 1. Search tree for the first move, two plies deep. Dashed nodes are pruned by Alpha-Beta because they cannot change the value already found for their parent.

A. Working Case 1: thief wins

On this map, the two guards start far enough from the artifact that the thief can reach it and return to the exit before being intercepted. At search depth 6, the thief wins after 52 turns, while Alpha-Beta expands 17,174 nodes to make its decisions across the whole game. Because the search only looks 6 plies ahead, the thief's early moves are not fully committed to the shortest path, and some back and forth movement is visible before it settles on a route towards the exit. This behaviour is a direct and expected consequence of the depth limit, not an error in the search. Figure 2 shows the final state of this run, with the thief having reached the exit.

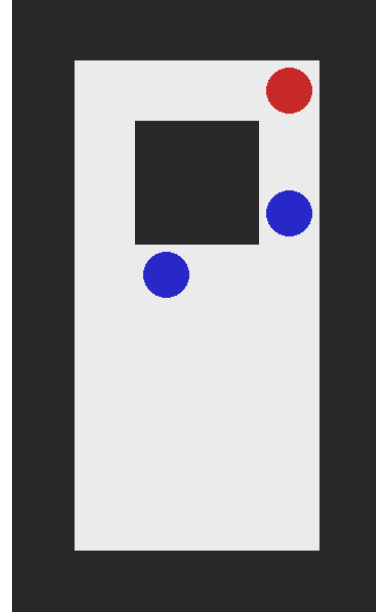


Fig. 2. Working Case 1, final state. The thief (red) has reached the exit (green) after collecting the artifact.

B. Working Case 2: guards win

On this map, the guards start close enough to the artifact that they reach the interception point before the thief can, regardless of search depth. At depth 6, the guards capture the thief after only 10 turns. This case confirms that a shorter guard to artifact distance reliably decides the outcome, independent of how far ahead either side can search. Figure 3 shows both guards closing in on the thief shortly before the capture.

C. Edge Case 1: thief with no safe move

This is a small sealed map where the thief starts in a dead end pocket with a guard on either side of the only entrance. The thief has exactly one legal move available, moving into the dead end, and is captured within four turns. This case demonstrates that Minimax still identifies and plays the least bad available move even when every outcome leads to a loss, rather than failing or behaving unpredictably when no winning move exists. Figure 4 shows the starting layout, with the thief

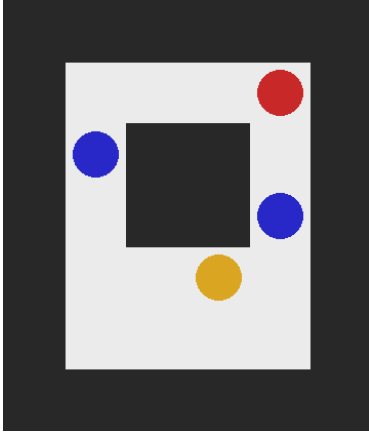


Fig. 3. Working Case 2. Both guards (blue) are closing in on the thief (red) shortly before it is captured.

sandwiched between the two guards and only the dead end pocket below open to it.

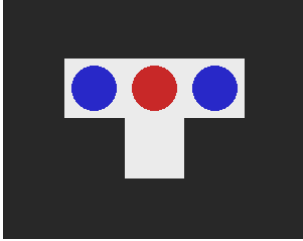


Fig. 4. Edge Case 1, starting layout. The thief (red) is boxed in between both guards (blue), with only the dead end pocket below open to it.

D. Edge Case 2: the horizon effect

This case uses a map with two parallel corridors of equal length joining a shared bottom corridor, with the artifact placed between them. With the same map and the same guard starting positions, a search of depth 2 results in the guards winning after 33 turns, while a search of depth 8 results in the thief winning after 94 turns. At depth 2, the thief cannot see far enough ahead to notice a guard converging on its intended path, and commits to a move that leads to capture two turns later. At depth 8, the same thief detects the same threat early enough to reroute and escape. This is a textbook horizon effect, where a search that is too shallow is blind to a danger just beyond its lookahead, and it is produced here with identical starting conditions, isolating search depth as the only variable that changes the outcome. Figure 5 shows the two outcomes side by side, the only difference between them is the search depth used by both the thief and the guards.

VIII. COMPLEXITY ANALYSIS

Depth limited Minimax has a time complexity of $O(b^d)$, where b is the branching factor and d is the search depth, since every reachable state up to the depth limit is expanded once. Its space complexity is $O(b \cdot d)$, since only the current path of

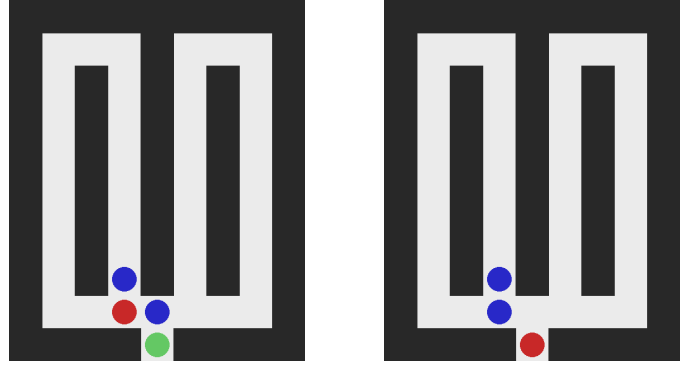


Fig. 5. Edge Case 2. Left, depth 2, both guards have cornered the thief right at the exit, moments before capture. Right, depth 8, the same map and guard start positions, the thief reaches the exit safely.

the tree needs to be kept in memory at any time. Alpha-Beta pruning does not change the worst case time complexity, but with good move ordering it approaches $O(b^{d/2})$ in practice, since roughly half of each ply can be skipped once a branch is shown to be irrelevant to the final decision.

In this system, the effective branching factor per agent is close to 5, since each agent can move in four directions or wait, minus any moves blocked by walls. Because three agents share the turn order, one full round of the game corresponds to three plies of the tree, so the useful lookahead in terms of full rounds is roughly one third of the raw depth value used in the code.

Table II shows nodes expanded and runtime for Minimax and Alpha-Beta on the same starting position, across increasing depth values.

TABLE II
NODES EXPANDED AND RUNTIME BY SEARCH DEPTH

Depth	Minimax	Alpha-Beta	MM time (s)	AB time (s)
2	13	9	0.000	0.000
3	40	20	0.001	0.000
4	121	57	0.001	0.001
5	418	144	0.004	0.002
6	1,507	383	0.005	0.002
7	4,774	678	0.023	0.007

The results follow the expected exponential growth pattern for Minimax, with the node count roughly tripling at each additional depth level. Alpha-Beta consistently expands fewer nodes at every depth, and the gap widens as depth increases, from a small difference at depth 2 to Alpha-Beta expanding about seven times fewer nodes than plain Minimax at depth 7. Both algorithms return identical scores and identical chosen moves at every depth tested, confirming that pruning only removes irrelevant branches and never changes the decision made. Figure 6 shows this growth visually, and Figure 7 shows the corresponding wall clock runtime, which follows the same pattern.

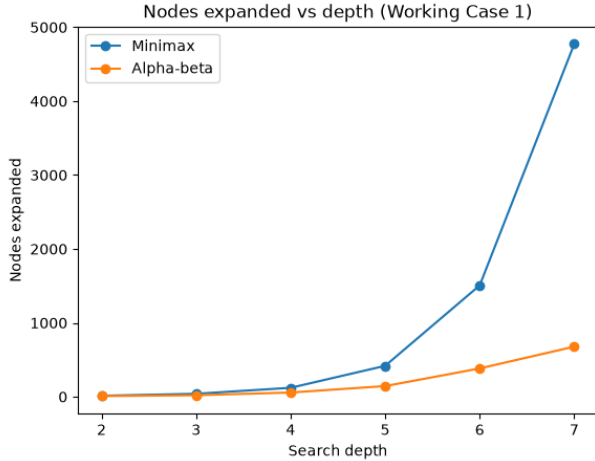


Fig. 6. Nodes expanded versus search depth, Minimax versus Alpha-Beta, on Working Case 1.

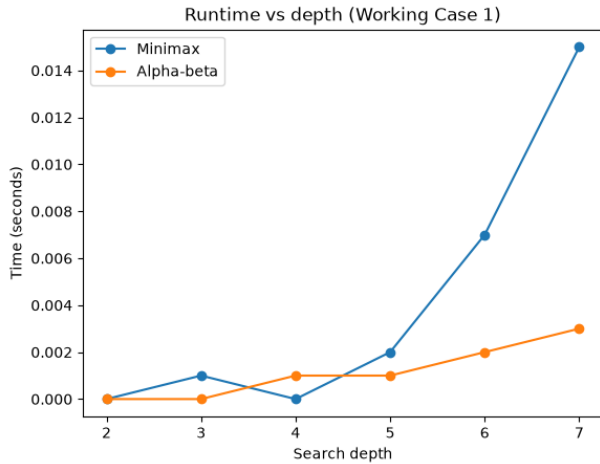


Fig. 7. Runtime in seconds versus search depth, Minimax versus Alpha-Beta, on Working Case 1.

IX. COMPARISON AND CONCEPTUAL UNDERSTANDING

A. Alpha-Beta versus greedy search

A one-step greedy search, where each agent looks only at its immediate neighbouring states and picks the best one with no further lookahead, was implemented as an alternative technique. Table III compares Alpha-Beta against greedy search across all four required cases.

TABLE III
ALPHA-BETA VERSUS GREEDY SEARCH ACROSS ALL CASES

Case	AB result	AB turns	Greedy result	Greedy turns
Working Case 1	Thief wins	52	Guards win	24
Working Case 2	Guards win	10	Guards win	8
Edge Case 1	Guards win	4	Guards win	5
Edge Case 2, depth 2	Guards win	33	Thief wins	34
Edge Case 2, depth 8	Thief wins	94	Thief wins	34

On Working Case 1, the greedy thief loses where the Alpha-Beta thief wins. With no lookahead, the greedy thief repeatedly oscillates between two neighbouring cells that look equally attractive one move at a time, making no real progress before it is caught. This illustrates a key limitation of greedy search, it produces fast decisions but cannot recognize when an apparently good immediate move leads to a worse long term outcome.

One result in Table III is worth noting directly. On Edge Case 2 at depth 2, greedy search actually wins where Alpha-Beta at the same depth loses. This happens because the shallow Alpha-Beta search commits early to a path based on an incomplete two step lookahead, and cannot later revise a decision that already looked locally reasonable, while the greedy thief simply reacts fresh each turn and, in this particular layout, avoids the same trap by chance. This shows that a limited amount of lookahead is not always strictly better than none, since a shallow search can be confidently wrong in a way that a reactive search is not.

Figures 8 and 9 show the same comparison across all five runs, on a logarithmic scale for the node counts since they range from the low tens to the tens of thousands. Alpha-Beta consistently expands far more nodes than greedy in every case, which is expected since it is searching several moves ahead rather than one, but the number of turns taken does not always follow the same pattern, showing that more search effort does not automatically mean a longer game, it means a better informed decision at each step.

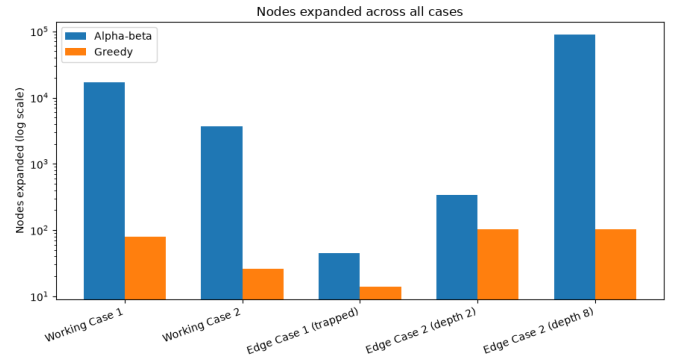


Fig. 8. Nodes expanded across all five case runs, Alpha-Beta versus greedy search, log scale.

B. Coordinated versus independent guards

The second comparison examines whether the two guards benefit from being coordinated, meaning both guards are MIN nodes inside one shared search tree, against an independent version, where each guard runs its own separate search against the thief and treats the other guard as a fixed obstacle rather than a planning partner. Both versions use the same underlying Minimax with Alpha-Beta search and the same evaluation function, so the only variable is whether the guards share one tree or each get their own. Table IV shows the results on Working Case 1.

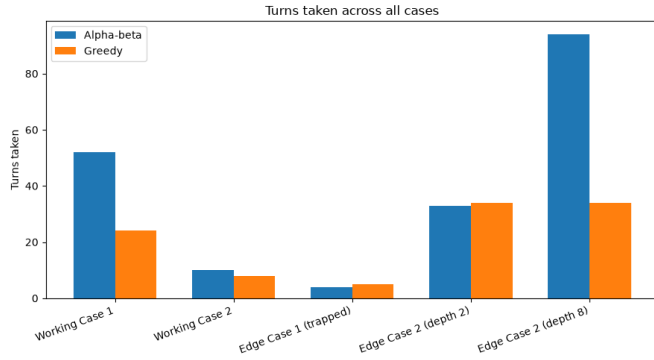


Fig. 9. Turns taken across all five case runs, Alpha-Beta versus greedy search.

TABLE IV
COORDINATED VERSUS INDEPENDENT GUARDS

Setup	Depth	Result	Nodes expanded
Coordinated	6	Thief wins	17,174
Independent	6	Guards win	42,521
Coordinated	9	Guards win	207,600

At equal nominal depth, the independent guards perform noticeably better than the coordinated team. This is because a shared search tree divides one depth budget across all three agents taking turns, so at depth 6 each individual guard effectively plans only about two of its own moves ahead, while an independent guard at the same nominal depth is not sharing that budget with a teammate, and effectively plans further ahead on its own moves. Increasing the coordinated search to depth 9 restores a guard win, showing that coordination can still be effective once it is given enough depth to make up for the budget it shares, though this comes at a substantial cost, requiring roughly five times as many nodes expanded as the independent version to reach a comparable outcome. This result highlights a genuine trade-off in multi-agent search, coordination is not automatically an advantage, it depends on how the available search depth is allocated across the agents that share it.

X. CONCLUSION

This case study designed and analyzed a small multi-agent adversarial system, a thief attempting a museum heist against two cooperating guards, using depth limited Minimax with Alpha-Beta pruning. The four required cases, produced with real simulation data, demonstrated normal play, a forced loss with no safe move, and a controlled horizon effect where identical starting conditions produced opposite outcomes purely from a change in search depth. Comparisons against a greedy baseline and an independently searching guard team showed that Alpha-Beta preserves decision quality while reducing search cost, that limited lookahead is not always better than none, and that coordinating agents inside one search tree costs effective depth per agent unless that depth is increased to compensate. Overall, the system demonstrates the intended AI

concepts through explainable, reproducible behaviour rather than through output alone.

REFERENCES

- [1] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Pearson, 2020.
- [2] D. E. Knuth and R. W. Moore, "An analysis of alpha-beta pruning," *Artificial Intelligence*, vol. 6, no. 4, pp. 293 to 326, 1975.